

# Verifiche indipendenti (spettro e stati) — trimero ad anello

Note di lavoro — Tesi triennale, Samuele Schiavi  
Relatore: Prof. Alessandro Chiesa

## Introduzione

Questo documento raccoglie i due script di verifica indipendente per il trimero ad anello: `verifica_anello.py` (spettro, campo critico, mappa dei segni — stesso principio di `verifica_catena.py` per la catena) e `verifica_stati_anello.py` (otto stati di Kambe espliciti, verificati come vettori — stesso principio di `verifica_stati_catena.py`). Entrambi sono script standalone, indipendenti dal modulo `trimer_ring_exact.py`: non lo importano, ricostruiscono l'Hamiltoniana e gli stati da zero con la propria implementazione, proprio per poter scovare un eventuale errore di convenzione che il modulo principale, controllando solo sé stesso, non potrebbe vedere.

A differenza dei due script gemelli della catena, qui vale la stessa nota già emersa nel confronto fra i due file: nonostante il nome simmetrico (`verifica_anello.py` vs `verifica_stati_anello.py`), i due script hanno scopi diversi — il primo lavora solo con lo spettro (mai costruisce un vettore di stato esplicito), il secondo lavora solo con gli stati (mai calcola un campo critico).

## `verifica_anello.py`

Script di controllo indipendente dal modulo principale (fino a poco fa chiamato `verifica.py`): spettro completo su 400 configurazioni casuali di  $(J, J', b)$ , energie dei tre multipletti a  $b = 0$ , campo critico confrontato fra formula analitica e scansione numerica diretta, mappa dei segni su una griglia  $61 \times 61 = 3721$  punti nel piano  $(J, J')$ , coordinate dei marker usati nelle figure.

### Funzioni interne.

- `kron3(a, b, c)` — prodotto di Kronecker di tre matrici  $2 \times 2$ ; costruisce operatori a 3 qubit a partire da operatori di singolo qubit.
- `op_on(site, P)` — colloca l'operatore di Pauli  $P$  sul sito indicato (1, 2 o 3), identità sugli altri due.
- `sigdot(i, j)` — il prodotto scalare di spin  $\sigma_i \cdot \sigma_j = X_i X_j + Y_i Y_j + Z_i Z_j$  come matrice  $8 \times 8$ .
- `H_trimer(J, Jp, b)` — l'Hamiltoniana completa dell'anello,  $J \sigma_1 \cdot \sigma_2 + J'(\sigma_2 \cdot \sigma_3 + \sigma_3 \cdot \sigma_1) + b \sum_i Z_i$ .
- `analitico(J, Jp, b)` — le otto energie attese dalla formula chiusa (non dalla diagonalizzazione), etichettate per blocco  $(A, B, C)$  e  $M$ .
- `bc_formula(J, Jp)` — campo critico  $b_c$  da formula analitica (o `None` se non esiste un incrocio per quella combinazione di segni).
- `bc_numerico(J, Jp, bmax, n)` — campo critico trovato scansionando  $b$  e cercando il salto di  $\langle S_z \rangle$  del fondamentale, raffinato per bisezione.
- `gs_formula(J, Jp)` — quale blocco (o quali, in caso di degenerazione) è il fondamentale a  $b = 0$ , confrontando direttamente le tre energie.

- `gs_regione_pdf(J, Jp)` — lo stesso, ma usando le disuguaglianze geometriche enunciate nel documento di teoria (regioni nel piano  $(J, J')$ ), per verificare che le due descrizioni (energie vs regioni) coincidano.

**Struttura dello script (5 blocchi di test, eseguiti in sequenza).**

1. Spettro numerico vs formula analitica su 400 punti  $(J, J', b)$  casuali.
2. Energie dei tre multipletti a  $b = 0$  su 8 combinazioni  $(J, J')$  scelte a mano.
3. Campo critico: formula vs scansione numerica, su 10 combinazioni.
4. Mappa dei segni: confronto energie vs regole geometriche, su una griglia  $61 \times 61$ .
5. Coordinate dei marker e copertura verticale delle rette nei 4 pannelli della figura di teoria — verifica che nessun marker o retta sia tagliato fuori scala.

**Come si usa.** Nessun argomento da riga di comando: si esegue con `python3 verifica_anello.py` e stampa i risultati dei 5 test a schermo, ognuno con un esito (OK/FALLITO/DA CONTROLLARE). Non richiede altri file del progetto (nessun `import` locale) — è completamente autosufficiente.

**Verifiche.** Rieseguito in questa sessione: errore massimo sullo spettro  $< 10^{-10}$  su 400 punti, campo critico coerente formula/numerico su tutti i 10 casi, 12 disaccordi su 3721 punti nella mappa dei segni (casi di confine già noti, alle frontiere fra regioni dove più blocchi sono degeneri — non un errore, coerente con quanto già osservato in sessioni precedenti).

**Conclusione.** Complementare, non ridondante, rispetto ai self-test interni di `trimer_ring_exact.py`: un controllo così esteso (migliaia di punti) serve specificamente a scovare eventuali errori di segno o di convenzione che un pugno di self-test mirati potrebbero non catturare.

## `verifica_stati_anello.py`

Script di controllo indipendente che costruisce esplicitamente gli **otto stati di Kambe** in base computazionale (non solo lo spettro) e li verifica uno per uno: norma, energia (numerica contro teorica), residuo come autovettore esatto, e ortogonalità reciproca (matrice di Gram). Scritto in questa sessione, mirror strutturale di `verifica_stati_catena.py` — stesso impianto, stessa sequenza di stampe, fisica dell'anello al posto di quella della catena.

**Funzioni interne.**

- `op(site, P)` — stessa funzione di `op_on` in `verifica_anello.py`, nome più corto per coerenza con lo script gemello della catena.
- `dot(i, j)` — stessa funzione di `sigdot`.
- `H_ring(J, Jp, b)` — la stessa Hamiltoniana di `H_trimero`, ricostruita indipendentemente (nome diverso, stesso risultato).
- `ket(bits)` — lo stato della base computazionale corrispondente a una stringa di 3 bit (es. '011'), come vettore  $\mathbb{C}^8$ .
- `state(coeffs)` — costruisce una sovrapposizione pesata a partire da una lista di coppie (ampiezza, stringa di bit); restituisce il vettore e la sua norma (non normalizzato automaticamente, la norma va controllata a parte — è esattamente uno dei test fatti più sotto).
- `E_teor(mult, M, J, Jp, b)` — l'energia teorica del blocco `mult` ("A", "B" o "C") al numero quantico  $M$ , dalla formula chiusa  $E(S_{12}, S, M) = 2(J - J')S_{12}(S_{12} + 1) + 2J'S(S + 1) - 3J - \frac{3}{2}J' + 2bM$ .

### Dati interni (non funzioni).

- **states** — dizionario che mappa (blocco, M) alla lista di coefficienti espliciti dell'autostato corrispondente, trascritti da `derivazione_stati_kambe_trimero.pdf` (Tabella 1).
- **QN** — dizionario che mappa ogni blocco alla coppia  $(S_{12}, S)$ .

**Come si usa.** Nessun argomento, nessun import locale: `python3 verifica_stati_anello.py`. Il punto di lavoro di test è fissato nello script ( $J = 1.3$ ,  $J' = -0.7$ ,  $b = 0.9$  — scelto senza simmetrie accidentali, per non mascherare un eventuale errore). Stampa, per ciascuno degli otto stati: norma al quadrato, energia numerica ed energia teorica affiancate, poi il residuo  $\|Hv - Ev\|$ ; infine il massimo elemento fuori diagonale e il massimo scarto dalla diagonale della matrice di Gram (per la verifica di ortonormalità).

**Verifiche.** Eseguito in questa sessione: tutti gli otto stati con norma  $^2 = 1.000000000$ , energia numerica identica a quella teorica alla quinta cifra decimale stampata, residuo fra 0 e  $6.3 \times 10^{-16}$ , matrice di Gram diagonale a  $2.2 \times 10^{-16}$ . **Conferma incrociata non banale:** il punto di test scelto qui ( $J=1.3, J'=-0.7, b=0.9$ ) coincide con quello già usato nella verifica numerica interna di `derivazione_stati_kambe_trimero.pdf`, e i valori di energia numerica ottenuti qui (es.  $A, +3/2 \rightarrow E = 2.60000$ ) coincidono esattamente con quelli già pubblicati in quel documento — una conferma indipendente, non solo interna a questo script.

**Conclusione.** Colma un buco reale identificato in questa sessione: per l'anello, la verifica degli stati di Kambe come vettori espliciti esisteva già, ma solo incorporata nei self-test di `trimer_ring_exact.py` — non come script standalone e indipendente. Ora la coppia `verifica_anello.py` / `verifica_stati_anello.py` è simmetrica, di nome e di contenuto, alla coppia già esistente per la catena.

## Conclusione generale

I due script coprono aspetti complementari e non sovrapposti: spettro/campo critico/simmetrie da un lato, stati espliciti come vettori dall'altro. Insieme formano, per l'anello, la stessa coppia di verifica indipendente già disponibile per la catena (documento gemello), completando così la simmetria fra le due topologie anche su questo fronte.